ЗВІТ
до практичної роботи №38 (П17)

Тема: Отримання даних з інтернету: HTTP-запити та відкриті API

Виконала: Вовк Марія Віталіївна
Група: І-23
Місто: Володимир

Мета роботи

Навчитися виконувати HTTP-запити за допомогою requests, передавати параметри, перевіряти статус відповіді, працювати з JSON, обробляти мережеві помилки та використовувати кеш. Для практичної використано відкриті API Національного банку України та Open-Meteo.

Підготовка

Створено каталог lab38, файли завдань task1.py–task5.py, requirements.txt та каталог data з файлом expenses.csv з практичної роботи 22. Загальна сума витрат становить 1248,00 грн. Для міста Володимир використано координати 50.842173, 24.3191015.

Завдання 1. Перший HTTP-запит

Програма task1.py надсилає GET-запит до API НБУ з параметром json, встановлює timeout=10, перевіряє код відповіді через raise_for_status(), читає JSON і виводить статус, Content-Type, кількість валют, курси USD та EUR і дату встановлення курсу.

Фактичні курси та кількість валют залежать від відповіді НБУ на момент запуску. Після запуску потрібно додати скриншот консолі.

Завдання 2. Витрати у валюті

Програма task2.py читає особисті витрати з CSV, підсумовує їх і робить окремі запити до НБУ для USD та EUR за параметром valcode. Сума у валюті обчислюється діленням загальної суми у гривнях на офіційний курс НБУ. Результат містить дату курсу та загальну суму у трьох валютах.

Завдання 3. Погода у Володимирі

Програма task3.py надсилає запит до Open-Meteo з координатами міста, параметрами current=temperature_2m,wind_speed_10m та часовим поясом Europe/Kyiv. Виводяться температура та швидкість вітру з одиницями, отриманими у відповіді API.

Завдання 4. Обробка помилок

У task2.py реалізовано три режими перевірки. Вони запускаються окремими командами та навмисно створюють відповідні помилки. Для всіх випадків повідомлення надсилається до stderr, а програма завершується з кодом 1.

python task2.py --demo connection — імітація відсутності доступу до сервера / ConnectionError.

python task2.py --demo http — запит до неіснуючого шляху / HTTPError (404).

python task2.py --demo timeout — короткий таймаут 0.001 секунди / Timeout.

У звіт потрібно вставити три скриншоти фактичного запуску цих команд.

Завдання 5. Кеш відповіді

Програма task5.py зберігає відповідь НБУ та час отримання у cache/rates.json. Якщо від часу збереження минуло менше години, програма використовує кеш. Інакше надсилає новий запит і оновлює файл кешу. Для перевірки потрібно запустити програму двічі: перший раз має показати source: api (cache saved), наступний — source: cache.

Контрольні запитання

1. З чого складається HTTP-запит і HTTP-відповідь?
Запит містить метод, адресу, заголовки, за потреби параметри та тіло. Відповідь містить код стану, заголовки та тіло з даними або повідомленням про помилку.

2. Що означають коди 200, 404 і 500?
200 — запит успішний; 404 — ресурс не знайдено; 500 — внутрішня помилка сервера.

3. Навіщо використовувати params=, а не дописувати параметри вручну?
requests самостійно кодує параметри URL, правильно обробляє спеціальні символи та робить код зрозумілішим.

4. Чому кожен запит повинен мати таймаут?
Таймаут не дозволяє програмі зависнути надовго, якщо сервер не відповідає або мережа працює повільно.

5. Чим ConnectionError відрізняється від HTTPError?
ConnectionError виникає, коли не вдається встановити з'єднання. HTTPError виникає після отримання HTTP-відповіді з помилковим кодом, наприклад 404.

6. Навіщо кешувати відповіді API і які ризики має застарілий кеш?
Кеш зменшує кількість запитів і пришвидшує роботу. Ризик — дані можуть застаріти, тому потрібно перевіряти час збереження та оновлювати кеш.

Висновок

Під час роботи створено програми для отримання курсів валют НБУ, конвертації особистих витрат, отримання поточної погоди у Володимирі, обробки мережевих помилок і кешування JSON-відповіді. Практично використано бібліотеку requests та відкриті API.

Примітка: Значення курсів і погоди змінюються. У звіт необхідно додати власні скриншоти фактичного виводу програм після запуску на комп'ютері.